

VBD Sparse Articulation Solve

A technical explainer for the horde/vbd-sparse-articulation-main branch

Newton Physics Engine

June 2026

Contents

1	Background: simulating articulated robots	2
1.1	What is an articulation?	2
1.2	The VBD / AVBD rigid-body solver	2
1.3	Why the local solve is slow for stiff joints	3
2	The sparse articulation solve: key idea	3
3	Implementation: step by step	4
3.1	Step 1: build the sparse layout (offline, once at construction)	4
3.2	Step 2: accumulate contact terms (body-diagonal only)	5
3.3	Step 3: assemble the block-sparse system (per-articulation kernel)	6
3.3.1	3a: inertia diagonal blocks	6
3.3.2	3b: joint contributions	6
3.4	Step 4: block Cholesky factorisation	7
3.5	Step 5: block triangular solves (forward and back substitution)	7
3.6	Step 6: apply the position update	7
3.7	Step 7: update dual variables (AVBD multipliers and contact state)	8
4	Data flow summary	8
5	Contacts in the coupled system	8
6	Supported joint types	8
7	Performance and convergence	9
7.1	Residual reduction	9
7.2	Timing (CPU, 44-body Unitree G1)	9
8	How to use the new feature	9
9	Limitations and future work	10
10	Quick-reference: key files	10

1 Background: simulating articulated robots

1.1 What is an articulation?

In Newton (and robotics simulation in general) an *articulation* is a tree (or graph) of rigid bodies connected by *joints*. A humanoid robot, for example, is one articulation: the pelvis is the root body, and every limb segment is a child body attached to its parent by a joint (revolute, fixed, prismatic, etc.).

Each rigid body has 6 degrees of freedom (DOFs): 3 translational and 3 rotational. Its pose at time t is a transform $\mathbf{q} = (\mathbf{p}, \mathbf{r}) \in \mathbb{R}^3 \times \text{SO}(3)$.

A joint constrains the relative motion between a parent body a and a child body b . Typical joint types used in Newton are:

Joint	Constrained DOFs
Fixed	all 6 (no relative motion)
Revolute	5 (free rotation about one axis)
Prismatic	5 (free translation along one axis)
Ball	3 (free rotation about all axes)
D6	user-defined mix of linear/angular DOFs
Cable	slack-only: pulls when taut, free otherwise

Joints are enforced via penalty (spring-like) forces. A high stiffness makes the joint nearly rigid; a low stiffness allows compliance.

1.2 The VBD / AVBD rigid-body solver

Newton’s rigid-body solver uses *Augmented Vertex Block Descent* (AVBD), an iterative position-based method. At a high level:

1. Integrate forward to a tentative “inertia” pose $\mathbf{q}^*$ (explicit Euler from the previous velocity and gravity).
2. Iteratively correct $\mathbf{q}$ to satisfy joint and contact constraints via Newton steps.
3. Extract new velocities from $(\mathbf{q} - \mathbf{q}_{\text{prev}})/\Delta t$.

At each iteration the solver minimises a total energy:

$$E(\mathbf{q}) = \underbrace{E_{\text{inertia}}(\mathbf{q})}_{\text{stay near inertia pose}} + \underbrace{\sum_j E_j(\mathbf{q})}_{\text{joint penalties}} + \underbrace{\sum_c E_c(\mathbf{q})}_{\text{contact penalties}} \quad (1)$$

Because E is nonlinear, minimising it directly is expensive. Newton’s method instead approximates E near the current pose $\mathbf{q}$ as a quadratic — a bowl-shaped surface — and jumps to the bottom of that bowl in one step. The approximation is a Taylor expansion in a small pose update $\delta \in \mathbb{R}^6$ (3 translational + 3 rotational):

$$E(\mathbf{q} \oplus \delta) \approx E(\mathbf{q}) + \underbrace{\frac{\partial E}{\partial \mathbf{q}}}_{\mathbf{g}^\top} \delta + \frac{1}{2} \delta^\top \underbrace{\frac{\partial^2 E}{\partial \mathbf{q}^2}}_{\mathbf{H}} \delta. \quad (2)$$

Here $\mathbf{g} \in \mathbb{R}^6$ is the gradient (net force/torque pulling the body away from the minimum) and $\mathbf{H} \in \mathbb{R}^{6 \times 6}$ is the Hessian, which encodes the curvature — intuitively, the “effective stiffness” in

every direction. The bottom of the bowl is where the gradient of (2) with respect to δ vanishes, i.e. $\mathbf{g} + \mathbf{H}\delta = \mathbf{0}$, which gives the *Newton step*:

$$\mathbf{H}\delta = -\mathbf{g}. \quad (3)$$

This is just a linear system: given the current forces and stiffnesses, solve for the pose correction δ that would zero the forces if the bowl approximation were exact. The body pose is then updated as $\mathbf{q} \leftarrow \mathbf{q} \oplus \delta$ and the process repeats.

Each energy term contributes additively to both $\mathbf{g}$ and $\mathbf{H}$. The inertia term adds $m/\Delta t^2$ to the translational diagonal of $\mathbf{H}$ and $\mathbf{R}\mathbf{I}_B\mathbf{R}^\top/\Delta t^2$ to the rotational diagonal ($m = \text{mass}$, $\mathbf{I}_B = \text{inertia tensor in body frame}$, $\mathbf{R} = \text{current rotation matrix}$). Each penalty constraint $E_j = \frac{k}{2}\|\mathbf{C}\|^2$ contributes $k\mathbf{J}^\top\mathbf{J}$ to $\mathbf{H}$, where $\mathbf{J} = \partial\mathbf{C}/\partial\mathbf{q}$ is the constraint Jacobian (a 3×6 matrix mapping pose velocity to constraint velocity).

In the **local** (per-body diagonal) solve, each body i assembles and solves its own 6×6 system:

$$\mathbf{H}_i\delta_i = -\mathbf{g}_i, \quad (4)$$

treating every other body as fixed. It is fast — one 6×6 solve per body — but ignores cross-body coupling from joints.

1.3 Why the local solve is slow for stiff joints

Consider two bodies connected by a stiff fixed joint. The joint energy penalises any relative displacement $\mathbf{C}$ between the anchor points:

$$E_{\text{joint}} = \frac{k}{2}\|\mathbf{C}\|^2. \quad (5)$$

When k is large (e.g. 10^5) the energy landscape is very “stiff” in the direction that separates the anchors. The local solve moves body a toward body b , but does not simultaneously account for how body b should move in response. The result is poor convergence: many VBD iterations are needed to reach the constraint manifold, especially for chains and loops.

Benchmarks on synthetic 128-body fixed chains show that the local solve barely reduces residuals in 8 iterations, whereas the coupled solve achieves $\sim 2\times$ residual reduction. For high stiffness-ratio cases the improvement is up to $117\times$.

2 The sparse articulation solve: key idea

The key insight is: instead of solving each body’s 6×6 system independently, solve all bodies in an articulation *simultaneously*. Each body i now has a 6-vector of unknowns δ_i (the pose correction), and all of these are coupled into one linear system:

$$\underbrace{\begin{pmatrix} \mathbf{H}_{11} & \mathbf{H}_{12} & \cdots & \mathbf{H}_{1n} \\ \mathbf{H}_{21} & \mathbf{H}_{22} & \cdots & \mathbf{H}_{2n} \\ \vdots & & \ddots & \vdots \\ \mathbf{H}_{n1} & \cdots & \cdots & \mathbf{H}_{nn} \end{pmatrix}}_{\mathbf{H} \in \mathbb{R}^{6n \times 6n}} \underbrace{\begin{pmatrix} \delta_1 \\ \vdots \\ \delta_n \end{pmatrix}}_{\delta} = - \underbrace{\begin{pmatrix} \mathbf{g}_1 \\ \vdots \\ \mathbf{g}_n \end{pmatrix}}_{\mathbf{g}} \quad (6)$$

Each entry $\mathbf{H}_{ij} \in \mathbb{R}^{6 \times 6}$ is a *block*. The diagonal block $\mathbf{H}_{ii}$ collects all forces acting on body i alone: inertia, contacts, and the self-coupling of every joint at body i . An off-diagonal block $\mathbf{H}_{ij}$ ($i \neq j$) is non-zero only when bodies i and j share a joint; it encodes how moving body j

changes the force on body i through that joint. If two bodies have no joint between them, their off-diagonal block is exactly zero and need not be stored.

This is what *block-sparse* means: most off-diagonal blocks are zero (bodies in different limbs of a robot are not directly coupled), so only the non-zero blocks need to be stored and computed. The local solve is the extreme case of discarding *all* off-diagonal blocks.

To make this concrete, consider a 4-body chain $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ (root to tip), with joints connecting $(0, 1)$, $(1, 2)$, and $(2, 3)$. Each joint produces one off-diagonal block, giving a *tridiagonal* block structure (■ = non-zero block, · = zero):

$$\mathbf{H}_{\text{chain}} = \begin{pmatrix} \blacksquare & \blacksquare & \cdot & \cdot \\ \blacksquare & \blacksquare & \blacksquare & \cdot \\ \cdot & \blacksquare & \blacksquare & \blacksquare \\ \cdot & \cdot & \blacksquare & \blacksquare \end{pmatrix} \quad \mathbf{H}_{\text{local}} = \begin{pmatrix} \blacksquare & \cdot & \cdot & \cdot \\ \cdot & \blacksquare & \cdot & \cdot \\ \cdot & \cdot & \blacksquare & \cdot \\ \cdot & \cdot & \cdot & \blacksquare \end{pmatrix}$$

The right matrix is what the local solve computes: four independent 6×6 systems with no cross-body information.

Now consider a *closed-loop* articulation: add a joint between body 3 and body 0 (closing the chain into a ring). Two additional off-diagonal blocks appear. The system is solved via Cholesky factorisation $\mathbf{H} = \mathbf{L}\mathbf{L}^\top$, which eliminates variables one by one. Eliminating body 0 couples bodies 1 and 3 together, creating a *fill-in* block at position $(3, 1)$ that was zero in $\mathbf{H}$ but becomes non-zero in $\mathbf{L}$ (marked $\circ$):

$$\mathbf{H}_{\text{loop}} = \begin{pmatrix} \blacksquare & \blacksquare & \cdot & \blacksquare \\ \blacksquare & \blacksquare & \blacksquare & \circ \\ \cdot & \blacksquare & \blacksquare & \blacksquare \\ \blacksquare & \circ & \blacksquare & \blacksquare \end{pmatrix} \quad (\circ = \text{fill-in in } \mathbf{L}, \text{ zero in } \mathbf{H})$$

The implementation pre-computes exactly which blocks are non-zero — including fill-in — *once* at solver construction time, before the simulation loop starts. At runtime only those blocks are allocated and touched, keeping both memory and computation minimal.

3 Implementation: step by step

3.1 Step 1: build the sparse layout (offline, once at construction)

File: `rigid_sparse_articulation.py`

Before the simulation loop starts, Newton pre-computes the *sparsity pattern* of the block matrix $\mathbf{H}$ — i.e. which 6×6 blocks are non-zero.

Because $\mathbf{H}$ is symmetric positive definite (inertia and stiffness are always positive, so the energy bowl curves upward in every direction), Cholesky factorisation applies: $\mathbf{H} = \mathbf{L}\mathbf{L}^\top$ where $\mathbf{L}$ is lower-triangular. Since $\mathbf{H}$ is symmetric, the upper triangle is the mirror image of the lower triangle, so we only need to track and store the lower-triangular sparsity pattern. The pattern of $\mathbf{L}$ is computed symbolically via `_symbolic_cholesky_pattern`:

```
def _symbolic_cholesky_pattern(body_count, edges):
    rows = [{i} for i in range(body_count)]
    for a, b in edges:
```

```

    row, col = max(a,b), min(a,b)
    rows[row].add(col)                                # add the joint edge
for k in range(body_count):
    later = [i for i in range(k+1, body_count) if k in rows[i]]
    for i_index, i in enumerate(later):
        for j in later[:i_index]:
            rows[max(i,j)].add(min(i,j)) # fill-in from elimination
return [sorted(row) for row in rows]

```

What this does. Start with the lower-triangular part of $\mathbf{H}$: there is a block at (i, j) (with $i > j$) for every joint connecting bodies i and j . Then simulate Cholesky elimination: when body k is eliminated, any two bodies $i > k$ and $j > k$ that both had a block in column k become coupled, so a fill-in block (i, j) is created. For a simple chain (tree) with bodies numbered root-to-leaf, Cholesky produces no fill-in. For a loop (e.g. a ring), there may be fill-in.

The resulting data structures (stored as Warp GPU arrays) are:

Array	Meaning
<code>articulation_body_offsets</code>	Start/end index in the body list for each articulation.
<code>articulation_joint_offsets</code>	Start/end index in the joint list for each articulation.
<code>articulation_bodies</code>	Global body IDs, grouped by articulation.
<code>articulation_joints</code>	Global joint IDs, grouped by articulation.
<code>articulation_block_row_offsets</code>	CSR-style row pointers into the block value array.
<code>articulation_block_cols</code>	Column indices for each non-zero block.
<code>articulation_diag_slots</code>	Index of the diagonal block for each body-row.
<code>body_articulation_local</code>	Local (within-articulation) index of each body.

Three working arrays are also allocated (zeroed each iteration):

Array	Purpose
<code>values</code>	Block matrix entries $[\mathbf{H}_{ij}]$, one <code>mat66f</code> per non-zero block.
<code>rhs</code>	Right-hand-side $\mathbf{g}_i$, one <code>vec6f</code> per body.
<code>delta</code>	Solution δ_i , one <code>vec6f</code> per body.

3.2 Step 2: accumulate contact terms (body-diagonal only)

File: `solver_vbd.py`, method `_solve_rigid_body_iteration_block_sparse_joints`

Before assembling the coupled joint system, the solver accumulates contact forces and Hessians into per-body arrays using the existing contact kernels (`accumulate_body_body_contacts_per_body`, `accumulate_body_particle_contacts_per_body`). These write into:

- `body_forces` and `body_torques` — the contact gradient.
- `body_hessian_ll`, `body_hessian_al`, `body_hessian_aa` — the 3×3 sub-blocks of the contact Hessian ($\mathbf{H}_{\text{lin-lin}}$, $\mathbf{H}_{\text{ang-lin}}$, $\mathbf{H}_{\text{ang-ang}}$).

These are then merged into the *diagonal* blocks of the block-sparse matrix. Off-diagonal contact coupling (between two bodies in contact) is intentionally excluded — it would require extending the sparsity pattern dynamically with every collision event, which is expensive and unnecessary for typical contact scenarios.

3.3 Step 3: assemble the block-sparse system (per-articulation kernel)

File: `rigid_sparse_articulation_kernels.py`, `kernel_solve_articulation_sparse_cpu`

This is the main Warp kernel. It is launched with one thread per articulation, so each articulation is processed independently (and therefore in parallel). Each thread does the following.

3.3.1 3a: inertia diagonal blocks

For each body i in the articulation, the kernel adds the inertia contribution to its diagonal block $\mathbf{H}_{ii}$:

$$\mathbf{H}_{ii}^{\text{inertia}} = \begin{pmatrix} m/\Delta t^2 \mathbf{I}_3 + \mathbf{H}_{ll}^{\text{contact}} & (\mathbf{H}_{al}^{\text{contact}})^\top \\ \mathbf{H}_{al}^{\text{contact}} & \mathbf{R} \mathbf{I}_B \mathbf{R}^\top / \Delta t^2 + \mathbf{H}_{aa}^{\text{contact}} \end{pmatrix} \quad (7)$$

where m is the body mass, $\mathbf{I}_B$ is the inertia tensor in the body frame, $\mathbf{R}$ is the current rotation matrix, and Δt is the time step. A small regulariser (10^{-6}) is added to the angular diagonal to prevent singular matrices for bodies with near-zero inertia.

The right-hand side $\mathbf{g}_i$ is initialised with the inertia force and torque:

$$\mathbf{f}_i = \frac{m}{\Delta t^2} (\mathbf{p}_i^* - \mathbf{p}_i) + \mathbf{f}_i^{\text{contact}}, \quad (8)$$

$$\boldsymbol{\tau}_i = \frac{1}{\Delta t^2} \mathbf{R} \mathbf{I}_B \boldsymbol{\theta} + \boldsymbol{\tau}_i^{\text{contact}}, \quad (9)$$

where $\mathbf{p}^*$ is the inertia-integrated position and $\boldsymbol{\theta}$ is the rotation-vector from the current to the inertia rotation.

3.3.2 3b: joint contributions

For each joint in the articulation the kernel assembles both the force/torque gradient and the coupled Hessian blocks.

Linear (translational) constraint. Let $\mathbf{a}_p$ and $\mathbf{a}_c$ be the world-space anchor points on the parent and child bodies. The constraint violation is $\mathbf{C}_{\text{lin}} = \mathbf{a}_c - \mathbf{a}_p$. After projecting out the free directions with projector $\mathbf{P}$ (identity for fixed joints, complement of the free axis for revolute/prismatic joints), the force contributed to the RHS is

$$\mathbf{f}_{\text{lin}} = k_{\text{lin}} \mathbf{P} (\mathbf{C}_{\text{lin}} - \alpha \mathbf{C}_{\text{lin}}^0) + k_{d,\text{lin}} \mathbf{P} \dot{\mathbf{C}}_{\text{lin}} + \mathbf{P} \boldsymbol{\lambda}_{\text{lin}}, \quad (10)$$

where k_{lin} is the penalty stiffness, $k_{d,\text{lin}}$ is damping, $\boldsymbol{\lambda}_{\text{lin}}$ is the Augmented Lagrangian multiplier, and $\alpha, \mathbf{C}^0$ are stabilisation terms (AVBD warmstart).

The contribution to the Hessian is computed via the Jacobian $\mathbf{J}$ of $\mathbf{C}$ with respect to the 6-DOF body velocity. For a body with COM at $\mathbf{c}$ and anchor at $\mathbf{a}$, the Jacobian row maps body DOF d (linear: $d \in \{0, 1, 2\}$, angular: $d \in \{3, 4, 5\}$) to:

$$J_{d<3} = \pm P_{c,d}, \quad (11)$$

$$J_{d \geq 3} = \pm \sum_k P_{c,k} [\mathbf{r} \times]_{k,d-3}, \quad (12)$$

where $\mathbf{r} = \mathbf{a} - \mathbf{c}$ is the moment arm and $[\mathbf{r} \times]$ is the skew-symmetric matrix of $\mathbf{r}$. The Hessian block contribution for a pair (i, j) is $\mathbf{H}_{ij} += (k_{\text{lin}} + k_{d,\text{lin}}/\Delta t) \mathbf{J}_i^\top \mathbf{J}_j$.

Angular (rotational) constraint. Let $\boldsymbol{\kappa}$ be the *rotation vector* that aligns the child anchor frame to the parent anchor frame (relative to rest):

$$\boldsymbol{\kappa} = \text{axis-angle}(q_{\text{parent}}^{-1} q_{\text{child}} \cdot q_{\text{rest}}^{-1}). \quad (13)$$

The projected angular violation is $\mathbf{P}_{\text{ang}} \boldsymbol{\kappa}$. The angular Jacobian is $\mathbf{J}_{\text{ang}} \in \mathbb{R}^{3 \times 3}$, mapping body angular velocity to $\dot{\boldsymbol{\kappa}}$. The torque and Hessian are assembled analogously to the linear case.

Drive and limit terms. For actuated joints (revolute, prismatic, D6) the kernel also assembles the contribution of PD drives and joint limits. For a revolute joint with axis $\hat{\mathbf{a}}$:

$$\theta = \hat{\mathbf{a}}^\top \boldsymbol{\kappa} + \theta_{\text{rest}}, \quad \dot{\theta} = \hat{\mathbf{a}}^\top \dot{\boldsymbol{\kappa}}, \quad (14)$$

$$f_{\text{drive}} = k_{\text{drive}}(\theta - \theta_{\text{target}}) + k_{d,\text{drive}}(\dot{\theta} - \dot{\theta}_{\text{target}}). \quad (15)$$

This is assembled as a rank-1 contribution to the angular Hessian blocks.

3.4 Step 4: block Cholesky factorisation

After all contributions are assembled, the kernel performs an in-place block Cholesky factorisation of $\mathbf{H}$ following the standard left-looking algorithm. For column k (body k):

1. Update the diagonal block: $\mathbf{A}_{kk} \leftarrow \mathbf{H}_{kk} - \sum_{s < k} \mathbf{L}_{ks} \mathbf{L}_{ks}^\top$.
2. Compute the diagonal factor: $\mathbf{L}_{kk} = \text{chol6}(\mathbf{A}_{kk})$, a dense 6×6 Cholesky factorisation.
3. For each $i > k$ with a non-zero block at (i, k) : $\mathbf{L}_{ik} = (\mathbf{A}_{ik} - \sum_{s < k} \mathbf{L}_{is} \mathbf{L}_{ks}^\top) \mathbf{L}_{kk}^{-\top}$.

The 6×6 dense Cholesky (`_cholesky66`) uses a standard lower-triangular Cholesky with a small clamp (10^{-9}) on the diagonal to guard against near-singular blocks.

Block indices that are zero (no joint) are skipped using `_find_block_slot`, which searches the sparse column list.

3.5 Step 5: block triangular solves (forward and back substitution)

With $\mathbf{H} = \mathbf{L}\mathbf{L}^\top$, the Newton step is solved as:

1. **Forward substitution:** solve $\mathbf{L}\mathbf{y} = \mathbf{g}$ for $\mathbf{y}$, row by row:

$$\mathbf{y}_i = \mathbf{L}_{ii}^{-1} \left(\mathbf{g}_i - \sum_{j < i} \mathbf{L}_{ij} \mathbf{y}_j \right).$$

2. **Backward substitution:** solve $\mathbf{L}^\top \boldsymbol{\delta} = \mathbf{y}$ for $\boldsymbol{\delta}$, reverse order:

$$\boldsymbol{\delta}_i = \mathbf{L}_{ii}^{-\top} \left(\mathbf{y}_i - \sum_{j > i} \mathbf{L}_{ji}^\top \boldsymbol{\delta}_j \right).$$

Each $\mathbf{L}_{ii}$ solve is done with `_solve_lower66` / `_solve_upper66_from_lower`, simple 6×6 triangular solves.

3.6 Step 6: apply the position update

The solved $\boldsymbol{\delta}_i = (\Delta \mathbf{x}_i, \boldsymbol{\omega}_i) \in \mathbb{R}^6$ is applied to body i 's pose with under-relaxation $\alpha_r \in (0, 1]$ (default 0.65):

$$\Delta \mathbf{x}_i^{\text{relax}} = \alpha_r \Delta \mathbf{x}_i, \quad (16)$$

$$\mathbf{p}_i^{\text{new}} = \mathbf{p}_i + \Delta \mathbf{x}_i^{\text{relax}}, \quad (17)$$

$$\mathbf{r}_i^{\text{new}} = \exp(\alpha_r \boldsymbol{\omega}_i) \cdot \mathbf{r}_i, \quad (18)$$

where $\exp(\boldsymbol{\omega})$ converts a rotation vector to a quaternion.

The relaxation $\alpha_r < 1$ prevents overshoot when the contact Hessian is stale (contacts are only re-detected every few substeps), which can make the combined inertia+contact Hessian a poor approximation.

3.7 Step 7: update dual variables (AVBD multipliers and contact state)

After the position update, the standard AVBD dual-variable update kernels are called to update:

- $\boldsymbol{\lambda}_{\text{lin}}$, $\boldsymbol{\lambda}_{\text{ang}}$ — the Augmented Lagrangian multipliers for joints.
- Contact sticking flags, friction multipliers.

This is identical to the local solve path; the block-sparse and local solves differ only in Steps 3–6.

4 Data flow summary

Phase	What happens
Construction	Symbolic Cholesky pattern computed; sparse index arrays allocated.
Per substep	Contact kernels accumulate body-diagonal forces/Hessians.
Per VBD iteration	<code>solve_articulation_sparse_cpu</code> kernel (one thread per articulation): 1. Zero sparse buffers. 2. Fill diagonal blocks (inertia + contact Hessian). 3. Fill diagonal + off-diagonal blocks (joint terms). 4. Block Cholesky factorisation (in place). 5. Forward + backward substitution. 6. Apply relaxed pose update.
Post iteration	Dual updates ($\boldsymbol{\lambda}$, contact state).

5 Contacts in the coupled system

Contacts between two bodies (body-body) or between a body and a particle (body-particle) are included via their *diagonal* Hessian and force contributions only. They are assembled by the existing `accumulate_body_body_contacts_per_body` kernel before the joint kernel runs, and the results ($\mathbf{H}_{ll}^{\text{contact}}$, $\mathbf{H}_{al}^{\text{contact}}$, $\mathbf{H}_{aa}^{\text{contact}}$) are merged into the diagonal 6×6 block of each body.

This means contacts effectively couple with all joints in the articulation (via the off-diagonal blocks), but two bodies in contact with each other do not get an off-diagonal block between them from the contact. This is a deliberate design choice: the contact graph changes every substep and adding it to the sparsity pattern would require re-running the symbolic Cholesky. Results show this is sufficient for humanoid contact scenarios.

6 Supported joint types

The sparse kernel handles the following joints:

Joint	Linear part	Angular part
Fixed	Full 3D penalty	Full 3D penalty
Revolute	Full 3D penalty	Penalty on 2 perp. axes; drive on free axis
Prismatic	Penalty on 2 perp. axes; drive on free axis	Full 3D penalty
Ball	Full 3D penalty	(no angular penalty)
D6	Per-axis mix	Per-axis mix
Cable	One-sided tension along cable axis	Bend penalty with Dahl friction

Free-direction projection is done with the projector $\mathbf{P} = \mathbf{I} - \hat{\mathbf{a}}\hat{\mathbf{a}}^\top$ (identity minus the outer product of the free axis unit vector). This removes the free direction from the constraint force, so only the constrained DOFs are penalised.

7 Performance and convergence

7.1 Residual reduction

On 128-body synthetic chains (fixed or revolute joints) and loops, the sparse articulation solve achieves:

- 1.3–1.6× one-iteration residual reduction vs the local solve.
- $\sim 2\times$ residual reduction after 8 VBD iterations.
- 117× energy reduction on an 8-body fixed ring with a 10^6 -to-1 stiffness ratio (linear vs. angular stiffness).

Prismatic joints benefit the most from coupling (16–18× linear energy reduction); D6 joints show more modest gains (2.6× linear).

7.2 Timing (CPU, 44-body Unitree G1)

Solver mode	Mean time per iteration
Local (diagonal) solve	51.2 μs
Block-sparse solve (serial CPU)	73.1 μs

The coupled solve is $\sim 1.4\times$ slower per iteration on CPU. Because it converges faster, the net cost per unit of residual reduction is lower.

A CUDA block-sparse kernel (GPU) is noted as future work; the current implementation only runs on CPU.

8 How to use the new feature

Enable the sparse articulation solve by passing the argument

`rigid_articulation_solve="block_sparse_joints"`

to SolverVBD:

```
solver = SolverVBD(
    model,
    rigid_articulation_solve="block_sparse_joints",
    rigid_articulation_relaxation=0.65,    # default, tune if needed
```

```

    sim_substeps=10,
    sim_iterations=5,
)

```

Notes.

- Only available on CPU for now (`device="cpu"`).
- `rigid_articulation_relaxation` $\in (0, 1]$. Lower values (~ 0.5) improve stability when contacts are stale; higher values (1.0) give faster convergence when contacts are well-resolved.
- The baseline "local" mode remains the default.

9 Limitations and future work

- **CPU only.** CUDA block-sparse kernels are not yet implemented. The serial CPU kernel runs one articulation per thread; a CUDA version would need warp-level parallelism over bodies within an articulation.
- **No off-diagonal contact coupling.** Contacts between bodies of the same articulation only appear in diagonal blocks. In scenarios with self-contact (robot hitting itself) this may limit convergence.
- **Static sparsity pattern.** The symbolic Cholesky is computed once at construction. Adding/removing joints at runtime would require rebuilding the layout.
- **May require relaxation for nonsmooth contact.** Stiff contact with poor contact Hessian approximation can cause divergence at `relaxation = 1.0`.

10 Quick-reference: key files

File	Responsibility
<code>rigid_sparse_articulation.py</code>	Sparse layout construction (once at init).
<code>rigid_sparse_articulation_kernels.py</code>	Warp kernel: assembly, Cholesky, solve, update.
<code>solver_vbd.py</code>	Dispatches local vs. block-sparse solve.
<code>tests/test_vbd_sparse_articulation.py</code>	Tests: chain, loop, contacts, drives.
<code>reports/vbd_sparse_articulation/</code>	Benchmarks and validation scripts.